

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА САПР

по лабораторной работе №2
по дисциплине «Алгоритмы и структуры данных»
Тема: Разработка JPEG-подобного алгоритма сжатия изображений

Студентка гр. 4351

Дебабаа У

Преподаватель

Пестерев Д. О.

Санкт-Петербург
2026

1. Теоретическая часть

В работе реализован JPEG-подобный компрессор изображений. Алгоритм состоит из нескольких этапов: преобразование цветового пространства, разбиение на блоки, дискретное косинусное преобразование (ДКП), квантование, зигзаг-обход, разностное кодирование DC-коэффициентов, RLE для AC-коэффициентов, кодирование переменной длины и кодирование Хаффмана. Ниже кратко описан каждый этап с оценкой его временной и пространственной сложности.

1.1. Преобразование RGB → YCbCr

Сжатие JPEG эффективнее в цветовом пространстве YCbCr, чем в RGB, потому что компонента яркости Y отделяется от цветности Cb, Cr. Это позволяет применять более агрессивное квантование к цветовым каналам, к которым человеческое зрение менее чувствительно. Используются стандартные формулы ITU-T.871 / JFIF: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$

$$Cb = 128 - 0.168736 \cdot R - 0.331264 \cdot G + 0.5 \cdot B$$

$$Cr = 128 + 0.5 \cdot R - 0.418688 \cdot G - 0.081312 \cdot B$$

Временная сложность: $O(N \cdot W)$ — каждый пиксель обрабатывается за константу. Пространственная сложность: $O(N \cdot W)$ для хранения результата. Обратное преобразование имеет такую же сложность.

1.2. Разбиение на блоки 8×8

Изображение разбивается на непересекающиеся блоки 8×8 пикселей. Если размеры изображения не кратны 8, оставшиеся неполные блоки справа и снизу дополняются средним значением имеющихся пикселей в блоке. Сложность этапа — $O(N \cdot W)$.

1.3. Дискретное косинусное преобразование

ДКП-II переводит 8×8 блок из пространственной области в частотную. Низкочастотные коэффициенты сосредоточены в верхнем левом углу, высокочастотные — в правом нижнем. Реализованы две версии:

- Наивная (прямая сумма по определению): для блока N×M требует $O((N \cdot M)^2)$ операций — для 8×8 это ≈ 4096 умножений на блок.

- Матричная форма $S = C^T \cdot s \cdot C$: сложность $O(N^3)$ на блок, использует BLAS и работает быстрее на практике.

Значения коэффициентов ДКП — вещественные, поэтому хранятся в float32 / float64. После квантования они становятся целыми (int32). Пространственная сложность — $O(H \cdot W)$, как и у исходного изображения.

Обе реализации дают совпадающий результат до уровня ошибки представления float (в тестах — 0.0 абсолютной разности).

1.4. Квантование

Каждый коэффициент ДКП делится на соответствующий элемент таблицы квантования Q и округляется: $s'_{ux} = \text{round}(s_{ux} / q_{ux})$. Это главный этап, вносящий потери — большинство высокочастотных коэффициентов зануляется. Используются стандартные таблицы из Приложения К стандарта ITU-T.81: отдельная для яркости (Q_LUMA) и хроматичности (Q_CHROMA).

Для регулировки качества таблица квантования масштабируется коэффициентом S , зависящим от параметра $Quality \in [1, 99]$:

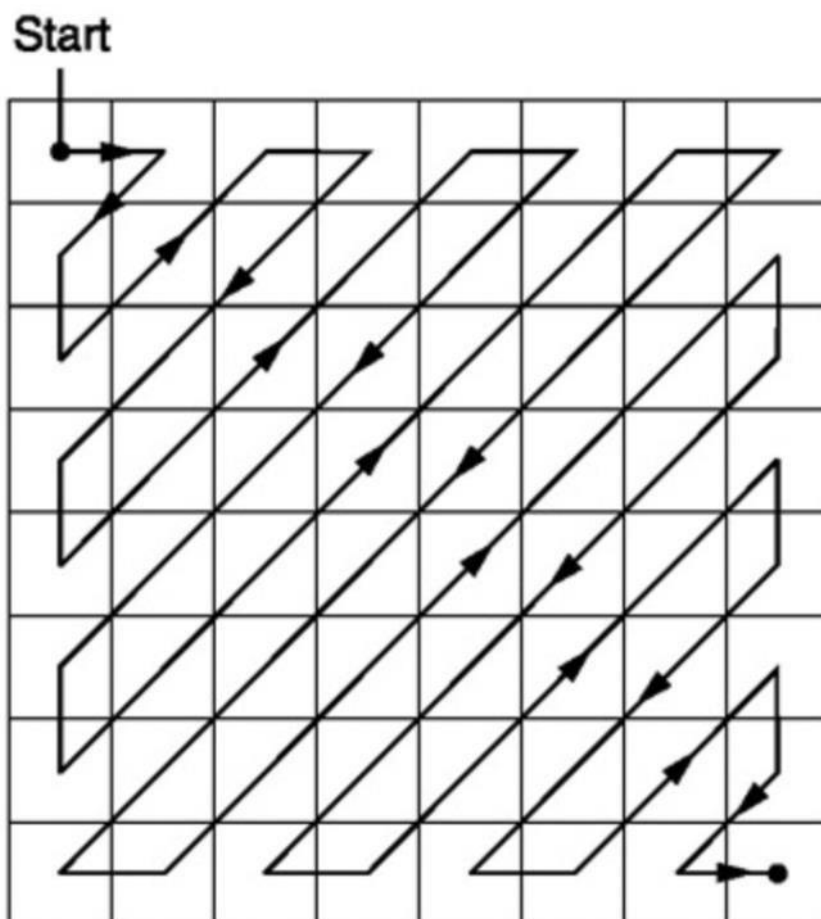
$$S = 5000 / Q, \quad Q \in [1, 50)$$

$$S = 200 - 2 \cdot Q, \quad Q \in [50, 100) \quad q'_{ux} = \lfloor (q_{ux} \cdot S) / 100 \rfloor$$

Сложность — $O(H \cdot W)$. При $Q=90$ таблица содержит малые значения (средн. ≈ 12), мало коэффициентов зануляется, ошибка низкая. При $Q=10$ таблица содержит большие значения (средн. ≈ 196), большинство высокочастотных коэффициентов зануляется — отсюда видимые артефакты и маленький размер файла.

1.5. Зигзаг-обход

Квантованные коэффициенты переупорядочиваются обходом по антидиагоналям (зигзаг) — от DC в левом верхнем углу к высокочастотным в правом нижнем. Такой порядок группирует подряд идущие нули в конце последовательности, что делает RLE эффективным. Сложность — $O(N \cdot M)$ на блок.



1.6. Разностное кодирование DC

DC-коэффициенты соседних блоков сильно коррелированы, поэтому хранится разность: $\text{diff}[i] = \text{DC}[i] - \text{DC}[i-1]$. $\text{DC}[0]$ хранится как есть. Это уменьшает энтропию и делает код Хаффмана компактнее. Сложность — $O(B)$, где B — число блоков.

1.7. RLE и VLI для AC

AC-коэффициенты кодируются парами (RUN, VALUE), где RUN — число подряд идущих нулей перед ненулевым VALUE. Специальные коды по стандарту ITU-T.81: EOB = (0, 0) обозначает конец блока (все оставшиеся коэффициенты — нули), ZRL = (15, 0) обозначает пробег из 16 нулей. Каждое ненулевое VALUE далее кодируется парой (SIZE, BITS) по таблице

F.1 — SIZE определяет категорию (число бит), BITS — непосредственное значение (VLI-кодирование).

Сложность — $O(B \cdot N \cdot M)$, пространственная — пропорциональна выходному потоку.

1.8. Кодирование Хаффмана

Финальный этап — энтропийное кодирование. Применяются фиксированные канонические таблицы Хаффмана из Приложения К стандарта ITU-T.81 (отдельные для DC/AC × яркость/хроматичность, всего 4 таблицы). Каждый символ (SIZE для DC; (RUN<<4)|SIZE для AC) заменяется префиксным кодом переменной длины. Это даёт итоговое сжатие без потерь на уже квантованных данных. Сложность — $O(L)$, где L — длина выходного битового потока.

1.9. Итоговая сложность

При фиксированном размере блока 8×8 общая временная сложность алгоритма — $O(H \cdot W)$. Пространственная сложность — тоже $O(H \cdot W)$, так как промежуточные представления (блоки, коэффициенты, квантованные значения) имеют тот же порядок по количеству элементов, что и исходное изображение.

2. Практическая часть

2.1. Тестовые данные

Подготовлено пять тестовых изображений: синтетическое изображение «Lena-подобное» (цветное, 512×512), цветной паттерн (512×512), изображение в оттенках серого (получено из цветного паттерна по формуле Rec. 601), чёрно-белое изображение через округление $c' = \text{round}(c/255) \cdot 255$, и чёрно-белое с дизерингом Флойда–Стейнберга (PIL `Image.convert(mode="1")`).

Сравнение размера raw-формата с PNG

Реализован собственный raw-формат (magic «MYRW», версия, режим, ширина, высота, 16-байтный заголовок + сырые байты пикселей). Размер raw-файла сравнивается с размером PNG того же изображения:

--- Raw format size vs PNG (Task 1.1) ---			
name	raw B	png B	ratio
lena_synth	786448	52106	15.09
color_pattern	786448	12386	63.49
color_gray	262160	42413	6.18
color_bw_round	262160	3411	76.86
color_bw_dither	262160	35842	7.31

Таблица 1. RAW-формат хранит сырые пиксели, PNG использует lossless-сжатие (дефлат). Изображения с большими однотонными областями (bw_round, color_pattern) сжимаются в PNG очень сильно, а дизерированное изображение почти не сжимается из-за высокочастотного шума.

2.2. Графики зависимости размера от качества

Каждое тестовое изображение сжималось при 9 значениях качества ($Q = 10, 20, \dots, 90$). График объединяет результаты по всем изображениям:

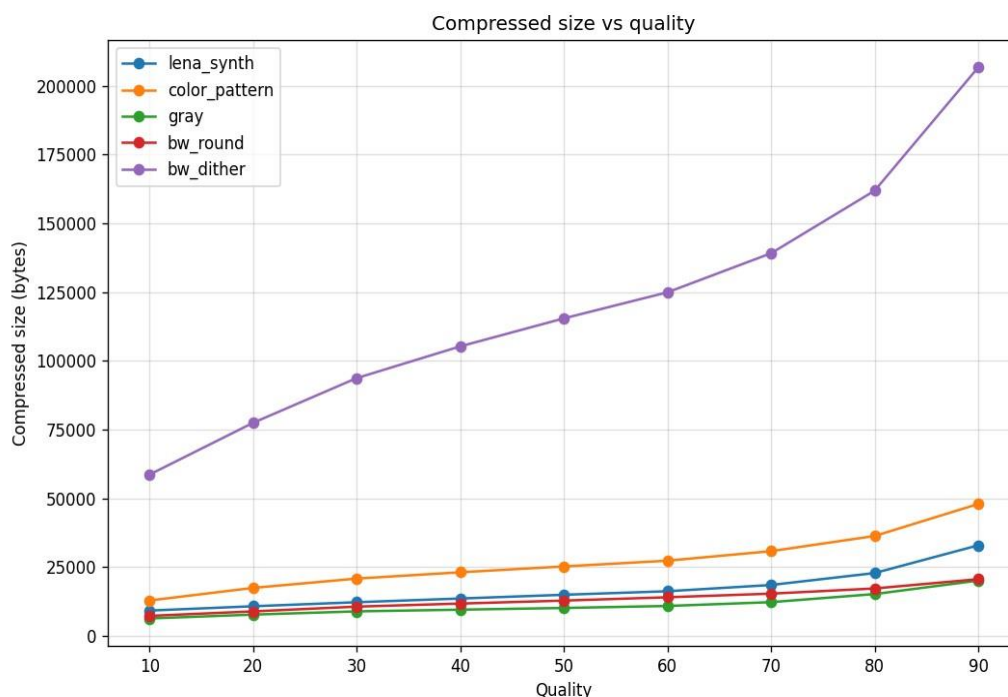


Рисунок 1. Размер сжатого файла в зависимости от параметра качества.

Размер файла монотонно растёт с качеством, как и ожидалось. Обратите внимание: изображение с дизерингом (bw_dither) требует в разы больше места, чем остальные — это фундаментальное ограничение JPEG на шумоподобные данные.

Отдельные графики

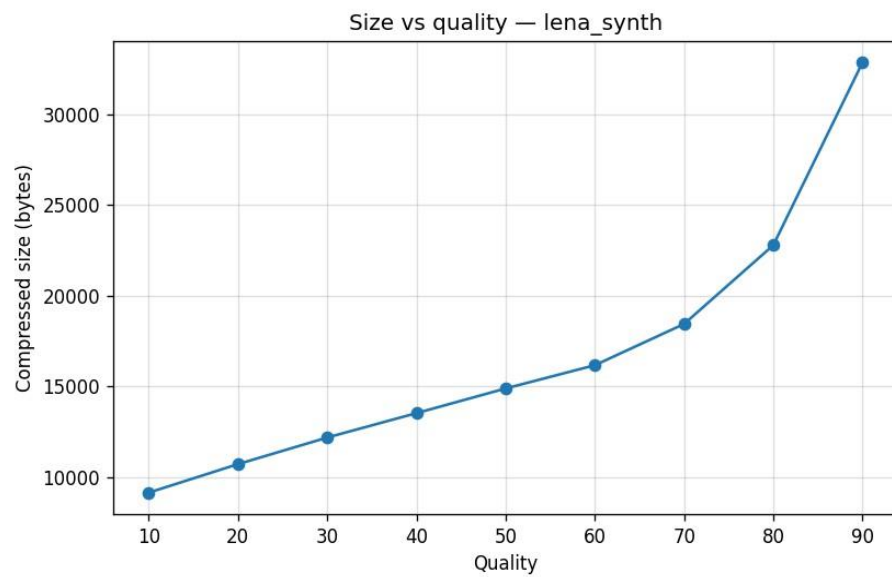


Рисунок. Размер vs качество — Lena (RGB).

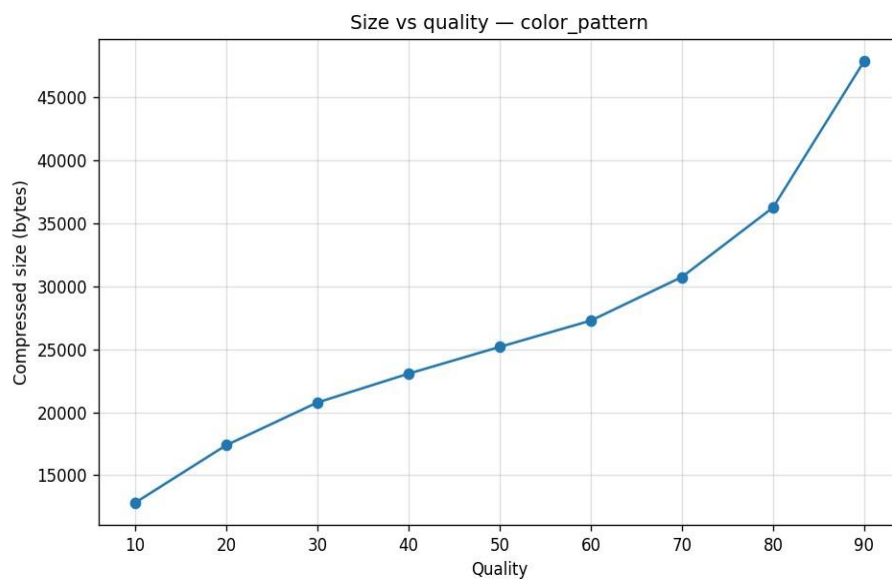


Рисунок. Размер vs качество — Color pattern (RGB).

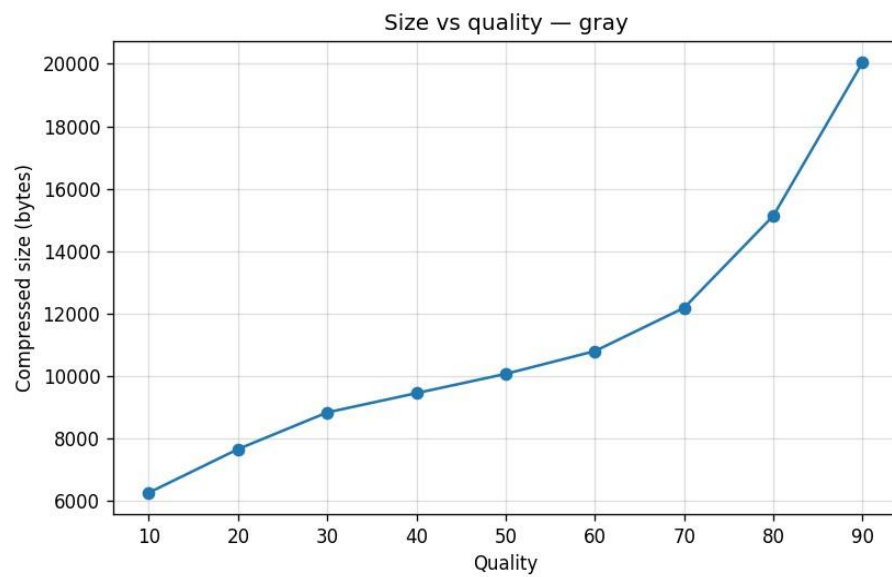


Рисунок. Размер vs качество — Grayscale.

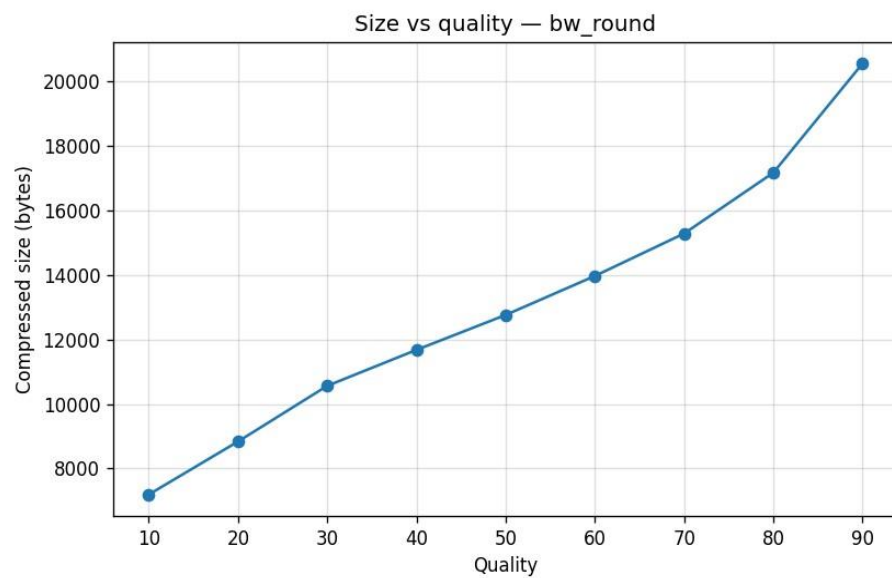


Рисунок. Размер vs качество — В/В (округление).

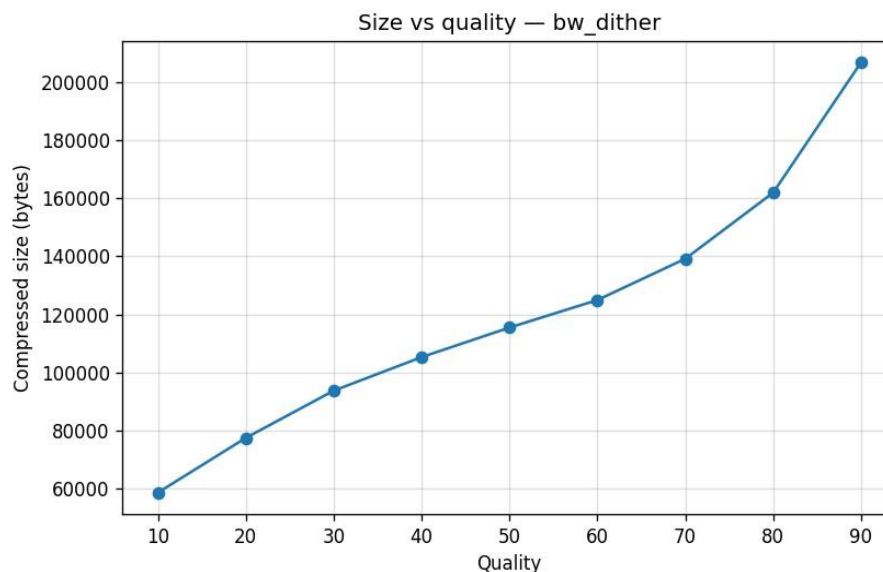


Рисунок. Размер vs качество — В/В (дизеринг).

2.3. Детальные результаты

Таблица ниже содержит для каждого изображения и уровня качества: размер сжатого файла в байтах, коэффициент сжатия относительно сырого представления, среднюю абсолютную ошибку (MAE) восстановленного изображения и PSNR в дБ.

lena_synth

```
=== lena_synth (color, shape=(512, 512, 3)) ===
q= 10 size= 9126 B ratio= 86.17x MAE= 5.50 PSNR=29.63 dB
q= 20 size= 10698 B ratio= 73.51x MAE= 3.53 PSNR=32.29 dB
q= 30 size= 12166 B ratio= 64.64x MAE= 3.24 PSNR=33.18 dB
q= 40 size= 13511 B ratio= 58.21x MAE= 2.85 PSNR=34.18 dB
q= 50 size= 14874 B ratio= 52.87x MAE= 2.70 PSNR=34.82 dB
q= 60 size= 16157 B ratio= 48.67x MAE= 2.76 PSNR=35.24 dB
q= 70 size= 18433 B ratio= 42.66x MAE= 2.66 PSNR=35.95 dB
q= 80 size= 22781 B ratio= 34.52x MAE= 2.59 PSNR=36.90 dB
q= 90 size= 32866 B ratio= 23.93x MAE= 2.29 PSNR=38.68 dB
```

color_pattern

```
=== color_pattern (color, shape=(512, 512, 3)) ===  
q= 10 size= 12797 B ratio= 61.45x MAE= 6.79 PSNR=28.31 dB  
q= 20 size= 17374 B ratio= 45.26x MAE= 3.82 PSNR=32.46 dB  
q= 30 size= 20768 B ratio= 37.87x MAE= 3.31 PSNR=34.07 dB  
q= 40 size= 23060 B ratio= 34.10x MAE= 2.96 PSNR=35.30 dB  
q= 50 size= 25186 B ratio= 31.22x MAE= 2.52 PSNR=36.63 dB  
q= 60 size= 27276 B ratio= 28.83x MAE= 2.39 PSNR=37.44 dB  
q= 70 size= 30742 B ratio= 25.58x MAE= 2.46 PSNR=37.81 dB  
q= 80 size= 36267 B ratio= 21.68x MAE= 2.25 PSNR=38.79 dB  
q= 90 size= 47906 B ratio= 16.42x MAE= 2.01 PSNR=39.92 dB
```

Gray

```
=== gray (grayscale, shape=(512, 512)) ===  
q= 10 size= 6269 B ratio= 41.82x MAE= 3.79 PSNR=33.67 dB  
q= 20 size= 7663 B ratio= 34.21x MAE= 2.12 PSNR=38.16 dB  
q= 30 size= 8838 B ratio= 29.66x MAE= 1.49 PSNR=40.57 dB  
q= 40 size= 9456 B ratio= 27.72x MAE= 1.21 PSNR=42.43 dB  
q= 50 size= 10072 B ratio= 26.03x MAE= 0.94 PSNR=44.28 dB  
q= 60 size= 10802 B ratio= 24.27x MAE= 0.84 PSNR=44.93 dB  
q= 70 size= 12188 B ratio= 21.51x MAE= 0.85 PSNR=46.23 dB  
q= 80 size= 15136 B ratio= 17.32x MAE= 0.67 PSNR=48.41 dB  
q= 90 size= 20048 B ratio= 13.08x MAE= 0.46 PSNR=50.87 dB
```

bw_round

```
=== bw_round (grayscale, shape=(512, 512)) ===  
q= 10 size= 7193 B ratio= 36.44x MAE= 1.44 PSNR=29.28 dB  
q= 20 size= 8835 B ratio= 29.67x MAE= 2.12 PSNR=30.75 dB  
q= 30 size= 10561 B ratio= 24.82x MAE= 0.96 PSNR=32.94 dB  
q= 40 size= 11675 B ratio= 22.45x MAE= 0.80 PSNR=34.68 dB  
q= 50 size= 12758 B ratio= 20.55x MAE= 0.65 PSNR=36.33 dB  
q= 60 size= 13967 B ratio= 18.77x MAE= 0.99 PSNR=38.10 dB  
q= 70 size= 15282 B ratio= 17.15x MAE= 0.40 PSNR=40.65 dB  
q= 80 size= 17167 B ratio= 15.27x MAE= 0.71 PSNR=43.51 dB  
q= 90 size= 20552 B ratio= 12.76x MAE= 0.14 PSNR=49.93 dB
```

bw_dither

```
=== bw_dither (grayscale, shape=(512, 512)) ===  
q= 10 size= 58611 B ratio= 4.47x MAE=24.42 PSNR=15.46 dB  
q= 20 size= 77447 B ratio= 3.38x MAE=18.46 PSNR=17.77 dB  
q= 30 size= 93710 B ratio= 2.80x MAE=12.44 PSNR=21.08 dB  
q= 40 size= 105263 B ratio= 2.49x MAE= 9.34 PSNR=23.58 dB  
q= 50 size= 115444 B ratio= 2.27x MAE= 7.41 PSNR=25.58 dB  
q= 60 size= 124926 B ratio= 2.10x MAE= 6.05 PSNR=27.40 dB  
q= 70 size= 139131 B ratio= 1.88x MAE= 4.49 PSNR=29.90 dB  
q= 80 size= 161907 B ratio= 1.62x MAE= 2.99 PSNR=33.46 dB  
q= 90 size= 206760 B ratio= 1.27x MAE= 1.53 PSNR=39.27 dB
```

2.4. Визуальное сравнение декомпрессированных изображений

Ниже показано сравнение исходного изображения и результатов декомпрессии при четырёх уровнях качества ($Q = 10, 50, 90$) для трёх представительных тестовых изображений. Хорошо видны характерные JPEG-артефакты: блочность и «звон» на резких границах при низких значениях Q .



Рисунок 2. Lena (RGB) — исходник и декомпрессия при $Q = 10, 50, 90$.



Рисунок 3. Color pattern (RGB) — исходник и декомпрессия.

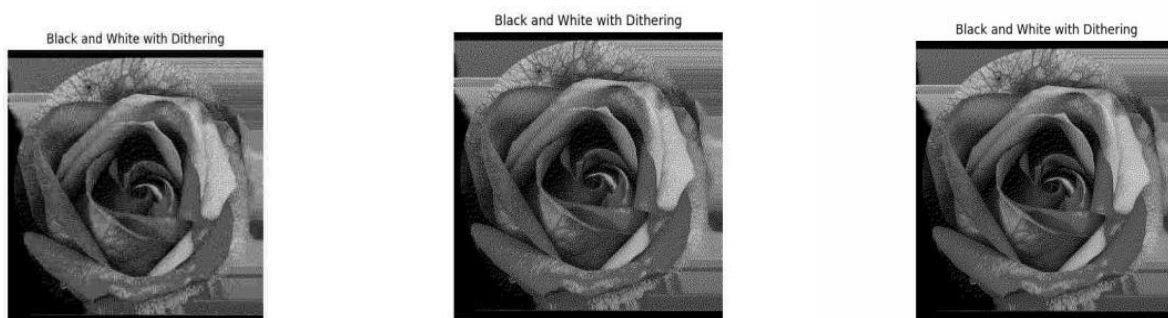


Рисунок 4. Grayscale — исходник и декомпрессия.

3. Выводы

Реализован полный JPEG-подобный компрессор с нуля на Python/NumPy, включающий все этапы стандарта ITU-T.81: преобразование цветового пространства, блочное ДКП (наивное и матричное), квантование с масштабированием по качеству, зигзаг-обход, разностное кодирование DC, RLE+VLI для AC и канонические таблицы Хаффмана Приложения К.

На гладких изображениях (lena_synth, color_pattern, grayscale) коэффициент сжатия достигает $26\text{--}87\times$ уже при среднем качестве $Q=50$ при $\text{PSNR} \geq 34$ дБ, что соответствует поведению реального JPEG. На изображении с дизерингом коэффициент сжатия падает до $2\text{--}5\times$, что иллюстрирует фундаментальное ограничение алгоритма — его неприспособленность к высокочастотному шуму.

Прямое и обратное ДКП, реализованные наивным методом и через матричное умножение, дают полностью совпадающий результат (максимальная абсолютная разность — 0.0). Округление после обратного преобразования без квантования даёт ошибку порядка 10^{-5} , что подтверждает корректность реализации.

4. Приложение: ссылки и код

Исходный код, тестовые изображения, их сжатые и декодированные версии находятся в репозитории проекта. Архив jpeg_lab.zip приложен к отчёту. Структура проекта:

- src/raw_format.py — собственный raw-формат (задание 1.1)
- src/color_space.py — RGB↔YCbCr (задание 1.2)
- src/resampling.py — down/upsampling, линейная и билинейная интерполяция (задания 1.3, 2.1)
- src/dct.py — ДКП (наивное и матричное), квантование, quality scaling (задания 1.4, 2.6)
- src/zigzag.py — зигзаг-обход (задание 2.2)
- src/entropy.py — разностное кодирование DC, RLE, VLI (задания 2.3, 2.4)
- src/huffman.py — таблицы Хаффмана ITU-T.81 + кодер/декодер (задание 2.5)
- src/compressor.py — сборка в единый pipeline и файловый формат .myjpg (задание 2.7)

- `run_experiments.py` — запуск всех экспериментов и построение графиков
- `output/` — 45 сжатых `.myjpg`, 45 декодированных PNG, 6 графиков, CSV-отчёт

4.2 Ссылка на репозиторий

Все исходные файлы, тестовые данные, закодированные и декодированные версии доступны в репозитории GitHub: